GAI / CASTROFF Prompt Documentation

SOEN 6011 – Software Engineering Processes | Delivery 3 (D3) | Function F3: sinh(x)

Arvind Lakshmanan (40310757) • Concordia University • Summer 2026

Purpose. This document records the generative-AI use for D3 Problems 7 and 8. It provides the prompt types, CASTROFF structure, prompt examples, and a concise explanation of how AI output was reviewed and verified. It supplements—rather than replaces—the real Flake8, Pylint, pdb, and PyUnit evidence.

1. D3 requirement and CASTROFF framework

The project requires every deliverable problem to use one or more publicly available GAI tools, with prompts based on CASTROFF, together with the prompt type, example prompt, and explanation of the output. For this D3 record, ChatGPT (OpenAI) was used as the GAI tool for review, refinement, and test-design support. The final technical claims were checked against the submitted source code and real tool evidence.

Element	Meaning	Use in this D3 record
C	**Constraints**	Boundaries, limitations, and rules the AI must respect.
A	**Audience**	The intended reader/user and their expected level of expertise.
S	**Structure**	How the response should be organized and sequenced.
T	**Tone**	The required communication style, e.g., academic and evidence-based.
R	**Role**	The professional role or perspective assigned to the AI.
O	**Output Format**	The requested delivery form: table, checklist, code, prose, etc.
F	**Focus**	The specific aspects of the task that deserve priority.
F	**Function**	The practical job the output must accomplish.

2. Problem 7 — Code quality, tooling, UIDP and accessibility

Prompt type: CASTROFF-structured analytical review and refinement prompt (zero-shot, constraint-driven).

CASTROFF breakdown

C — Constraints	Work only on F3 sinh(x). Preserve the D2 from-scratch Maclaurin approach, Tkinter GUI, exception handling, and supported input range −20 ≤ x ≤ 20. Do not use math.sinh, math.exp, factorial helpers, NumPy, or SciPy for the production calculation. Check PEP-8, Flake8, pdb, Pylint, Semantic Versioning, UIDP, and accessibility. Do not invent tool results.
A — Audience	SOEN 6011 professor/evaluator assessing D3 compliance and evidence.
S — Structure	Organize the response as: compliance audit → necessary code/GUI improvements → UIDP decision rationale → verification checklist.
T — Tone	Academic, precise, concise, evidence-based, and transparent about limitations.
R — Role	Act as a senior Python software engineer and usability/accessibility reviewer.
O — Output Format	A requirement-by-requirement table plus concrete revisions and commands/evidence that must be verified outside the GAI system.
F — Focus	D3 Problem 7 compliance for the actual F3 Tkinter calculator, not a redesign of the mathematical function.
F — Function	Review and improve the existing implementation so it is ready for D3 demonstration and poster evidence.

Example prompt used

Act as a senior Python software engineer and usability/accessibility reviewer. Review my SOEN 6011 Delivery 3 implementation for F3, sinh(x). The audience is my professor/evaluator, so keep the review academic, concise, and evidence-based. Preserve the existing from-scratch recurrence-based Maclaurin implementation and Tkinter GUI. The supported range must remain -20 <= x <= 20. The production calculation must not use math.sinh, math.exp, factorial helpers, NumPy, SciPy, or another mathematical library. Audit the code against D3 Problem 7: PEP-8 conformance, Flake8 evidence, pdb debugger evidence, Pylint static-analysis evidence, Semantic Versioning, UIDP selection by mind-map reasoning, and accessibility. Do not fabricate any Flake8, Pylint, pdb, or execution result; explicitly mark those as items that require real tool verification. Structure the response as (1) compliance status, (2) exact revisions needed, (3) UIDP principles that apply/do not strongly apply with reasons, and (4) a verification checklist. Output a compact table followed by actionable changes. Focus only on D3 readiness of this F3 calculator.

AI output and how it was used

- GAI was used to review the implementation against the D3 checklist and to identify concrete quality/usability improvements rather than to replace the required engineering tools.
- The final source reflects modular GUI helper methods, PEP-8-oriented naming/docstrings, Semantic Version 1.1.0, keyboard alternatives (Enter/Alt+C, Alt+L, Escape, and ordinary Tab navigation), textual status/error feedback, a read-only result area, clear labels, and a resizable Tkinter window.
- The UIDP review identified consistency, visibility, feedback, error prevention, simplicity, user control, accessibility, and keyboard-friendly interaction as applicable. Multi-user collaboration, complex customization, advanced personalization, and gesture-based input were treated as not strongly applicable to this single-user scientific calculator.
- AI did not determine whether Flake8/Pylint/pdb actually succeeded. Those claims were verified through the submitted terminal screenshots: Flake8 returned without listed violations; Pylint reported 9.86/10 with two minor warnings; pdb was used to step through calculate_sinh_from_scratch.

3. Problem 8 — Unit-test design and PyUnit coverage

Prompt type: CASTROFF-structured test-design and coverage-review prompt (zero-shot, requirements-driven).

CASTROFF breakdown

C — Constraints	Use Python unittest/PyUnit. Test the existing public helper/calculation functions without changing the production algorithm. Cover valid inputs, invalid inputs, finite-range boundaries, numerical behavior, and the odd-function property. Do not claim tests pass unless execution evidence confirms it.
A — Audience	SOEN 6011 evaluator reviewing Problem 8 test adequacy and traceability to the implementation.
S — Structure	Group tests by helper functions, input parsing, and sinh calculation; include normal, boundary, invalid, and property-based examples.
T — Tone	Technical, concise, objective, and suitable for an academic software-engineering submission.
R — Role	Act as a Python test engineer reviewing PyUnit coverage for a numerical GUI application.
O — Output Format	A test matrix and PyUnit-oriented recommendations naming the behavior, representative input, and expected outcome.

F — Focus	The numerical core and input-validation behavior of F3 sinh(x), including −20 and 20, NaN/infinity rejection, and sinh(−x) = −sinh(x).
F — Function	Produce and review a defensible unit-test set for D3 Problem 8 and identify coverage gaps before execution.

Example prompt used

> Act as a Python test engineer. I am preparing SOEN 6011 Delivery 3 Problem 8 for an F3 sinh(x) calculator implemented from scratch with a recurrence form of the Maclaurin series. The evaluator needs a defensible PyUnit/unittest test suite. Do not alter the production algorithm. The supported input range is -20 <= x <= 20. Design tests for helper functions, parsing, blank/non-numeric input, NaN/infinity, values just outside the range, zero, representative positive and negative values, both supported boundaries, and the odd-function property sinh(-x) = -sinh(x). Organize the tests into logical unittest.TestCase classes. For numerical tests, use explicit expected reference values and an appropriate tolerance. Do not claim the suite passes until it is actually executed. Output (1) a compact test matrix, (2) recommended PyUnit test cases, and (3) a checklist of missing coverage. Keep the tone technical and concise and focus only on D3 Problem 8.

AI output and how it was used

- The coverage review supported three logical groups used in the final test module: helper functions, input parsing, and numerical sinh calculation.
- The final suite contains 16 tests covering positive/negative absolute value behavior, NaN detection, infinity detection, accepted integer/decimal/scientific input, supported boundaries, blank/non-numeric/non-finite/out-of-range rejection, zero, known positive and negative sinh values, the full supported boundaries, and the odd-function property.
- Execution—not GAI—established the final result. The submitted PyUnit screenshot shows “Ran 16 tests” followed by “OK.”

4. Human verification and deliberate non-use of GAI

GAI was used as an assistant for structured review, refinement, and test-design reasoning. It was deliberately not used as a substitute for deterministic engineering evidence. The following evidence must come from the actual program or development tools:

- Flake8 output for PEP-8/style conformance.
- Pylint output and rating/warnings.
- pdb stepping/debugger session.
- PyUnit/unittest execution result.
- Tkinter GUI behavior during the in-person demonstration.

This separation reduces the risk of presenting generated claims as measured results. Any GAI recommendation was accepted only when it matched the D3 specification and the actual submitted source/evidence.

5. Final D3 GAI compliance summary

Item	Status	Evidence in this document
Problem 7	Yes	CASTROFF review/refinement prompt documented; output explanation included; real Flake8/Pylint/pdb evidence kept separate.
Problem 8	Yes	CASTROFF test-design prompt documented; output explanation included; final 16-test PyUnit result verified by execution evidence.
Prompt framework	Yes	Uses Constraints, Audience, Structure, Tone, Role, Output Format, Focus, Function.
Prompt type stated	Yes	Prompt type is explicitly identified for both D3 problems.
Example prompts included	Yes	Full example prompts are included for Problems 7 and 8.
Output explanation included	Yes	Each problem records what GAI contributed and what was verified manually/by tools.

References

1. Concordia University, Department of Computer Science and Software Engineering, SOEN 6011: Software Engineering Processes, Project Description, Summer 2026.
2. H. Tavakoli, Prompt Engineering for Everyone: A Self-Taught, Human-Centered Approach to AI Programming, 2026. CASTROFF is presented as Constraints, Audience, Structure, Tone, Role, Output Format, Focus, and Function.